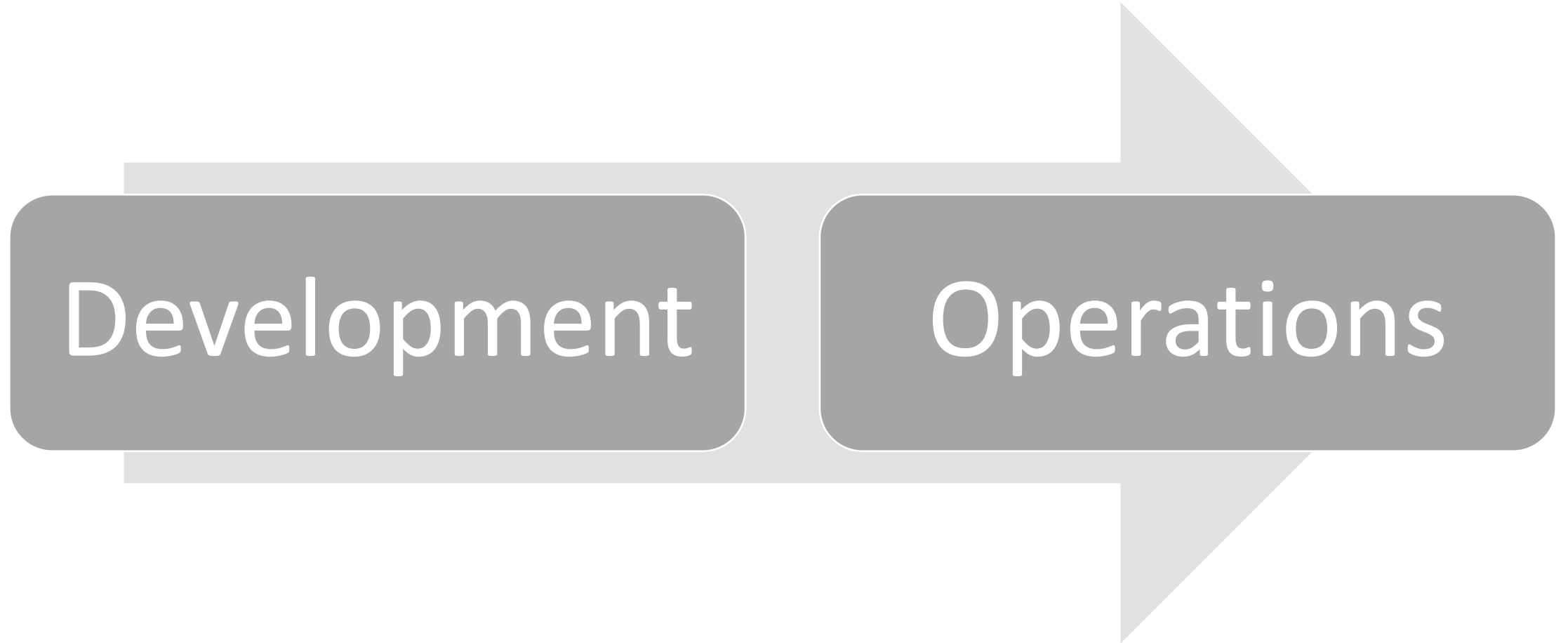




DevOps

1st Way: Technical Practices of Flow

Fast Flow





Continuous Delivery



Menti: What do we need to
have continuous delivery?

Continuous Delivery



Automated deployment pipelines



Automated tests



Integrate code into trunk daily



Architect environment and code to enable low-risk release

Main focuses

01

create the
foundation of
deployment
pipeline

02

enabling fast and
reliable automated
testing

03

enabling and
practicing
continuous
integration and
testing

04

automating,
enabling, and
architecting for
low-risk releases

Outcomes



reduce the lead time to get production-like environments



enable continuous testing that gives everyone fast feedback on their work



enable small teams to safely and independently develop, test, and deploy their code into production



makes production deployments and releases a routine part of daily work



Automated Deployment Pipelines

Always use production-like environments

Environments must be created in an automated manner

On demand

Using scripts and configuration information stored in version control

Entirely self-serviced

Without any manual work required from Operations

Enable On-Demand Creation of Dev, Test, and Production Environments

Developers run production-like environments on their own workstations

~~Specifications of the production environment in a document or on a wiki page~~

Create a common build mechanism that creates all of our environments

~~Requirements in documents or as knowledge in someone's head~~

Requirements are codified in automated environment build process

Use automation everywhere

copying a virtualized environment (e.g., a VMware image, running a Vagrant script, booting an Amazon Machine Image file in EC2)

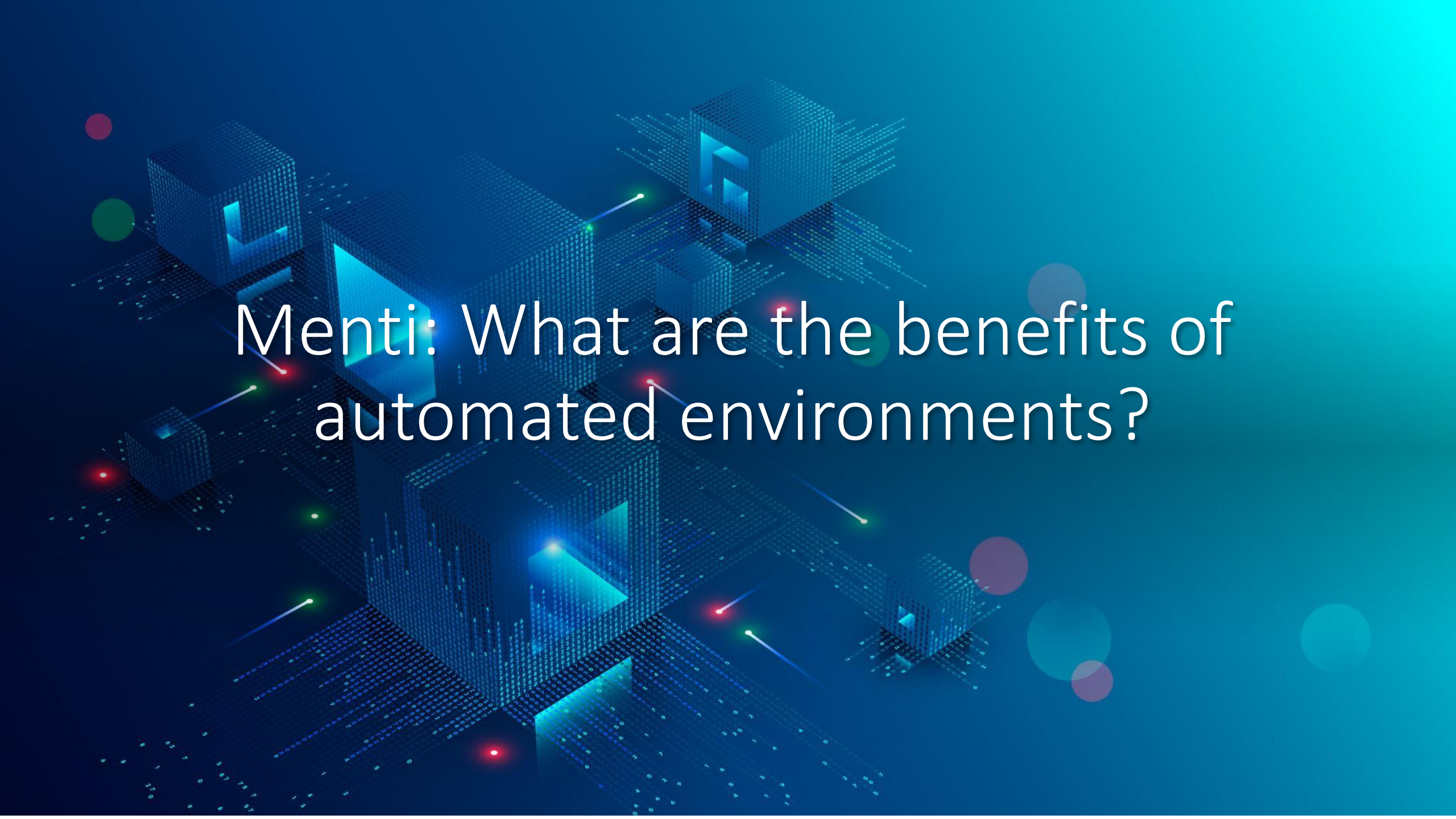
building an automated environment creation process that starts from “bare metal” (e.g., PXE install from a baseline image)

using “infrastructure as code” configuration management tools (e.g., Puppet, Chef, Ansible, Salt, CFEngine, etc.)

using automated operating system configuration tools (e.g., Solaris Jumpstart, Red Hat Kickstart, Debian preseed)

assembling an environment from a set of virtual images or containers (e.g., Docker, Kubernetes)

spinning up a new environment in a public cloud (e.g., Amazon Web Services, Google App Engine, Microsoft Azure), private cloud (for example, using a stack based on Kubernetes), or other PaaS (platform as a service, such as OpenStack or Cloud Foundry, etc.)

An abstract digital cityscape rendered in a dark blue and cyan color palette. The scene is composed of numerous 3D cubes and rectangular blocks of varying sizes, some of which are illuminated from within, casting a bright blue glow. The surfaces of these blocks are covered in a dense pattern of binary code (0s and 1s). Several thin, glowing lines in red, green, and blue extend from the blocks, suggesting data flow or connections. In the background, more blocks are visible, some with glowing rectangular openings. The overall composition is layered and three-dimensional, creating a sense of depth and complexity. The text "Menti: What are the benefits of automated environments?" is overlaid in the center in a white, sans-serif font.

Menti: What are the benefits of automated environments?

Benefits of automated environment creation



Quickly reproduce, diagnose, and fix defects



Safely isolated from production services and other shared resources



Experiment with changes to the environments, as well as to the infrastructure code that creates it



Shared knowledge between Development and Operations

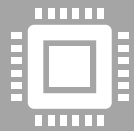
Single Repository of Truth for the Entire System

-
- all application code and dependencies
 - any script used to create database schemas, application reference data, etc.
 - all the environment creation tools and artifacts (e.g., VMware or AMI images, Puppet, Chef, or Ansible scripts.)
 - any file used to create containers (e.g., Docker, Rocket, or Kubernetes definitions or composition files)
 - all supporting automated tests and any manual test scripts
 - any script that supports code packaging, deployment, database migration, and environment provisioning
 - all project artifacts (e.g., requirements documentation, deployment procedures, release notes, etc.)
 - all cloud configuration files (e.g., AWS Cloudformation templates, Microsoft Azure Stack DSC files, OpenStack HEAT)
 - any other script or configuration information required to create infrastructure that supports multiple services (e.g., enterprise service buses, database management systems, DNS zone files, configuration rules for firewalls, and other networking devices)

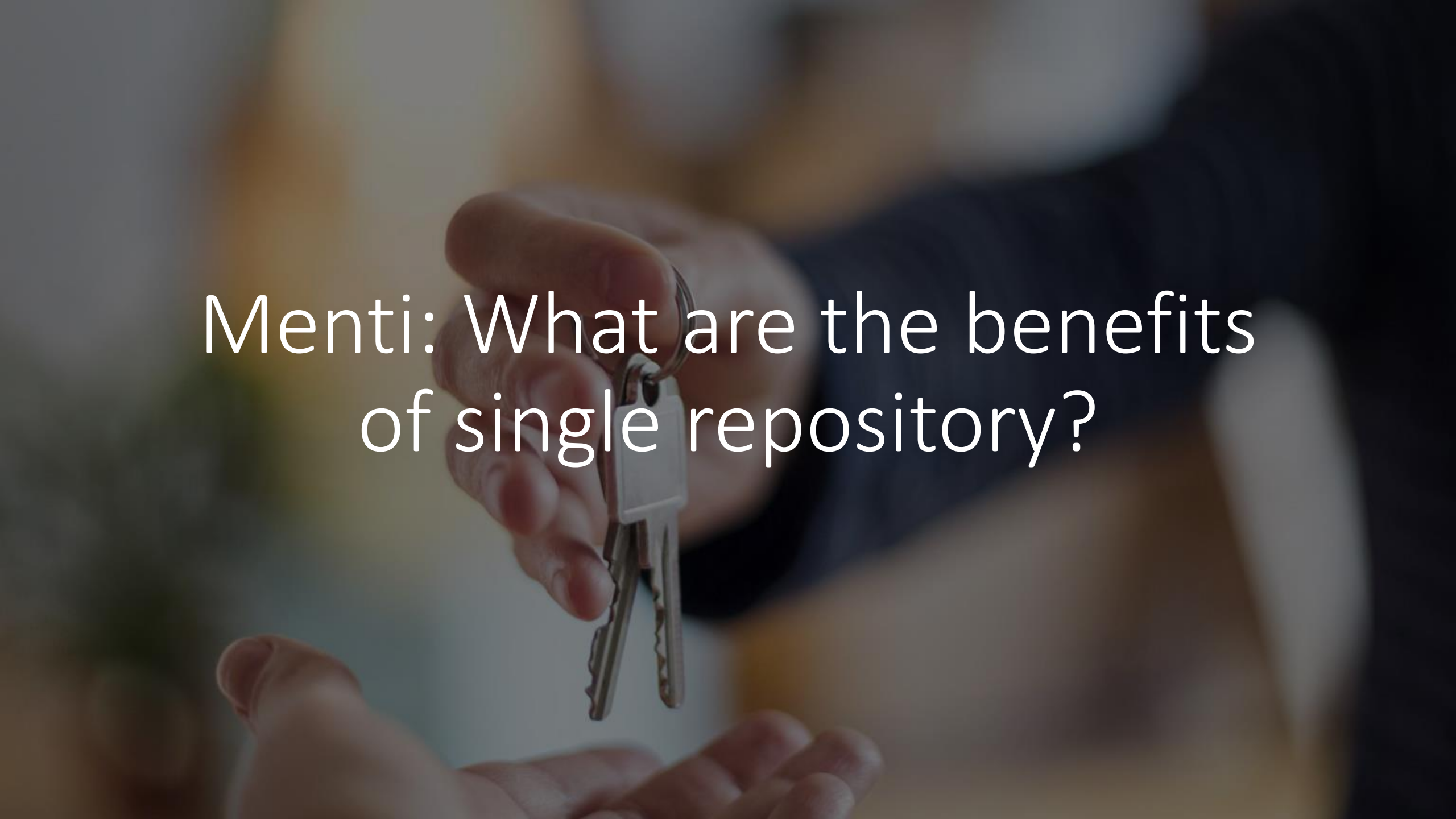
Single Repository - remarks



may have multiple repositories for different types of objects and services



Create and store a cryptographic hash of these objects at build time and validate this hash at deploy time to ensure they haven't been tampered with

A close-up photograph of a hand holding a set of keys, with another hand open below it. The background is blurred, showing a person in a blue shirt. The text is overlaid in white on the image.

Menti: What are the benefits
of single repository?

Single repository benefits



All application and environment changes are recorded in version control



Enables to see all changes that might have contributed to a problem and the means to roll back to a previous known, running state



There are orders of magnitude more configurable settings in our environment than in our code



Enables communication between Development, QA, Infosec, and Operations

Make Infrastructure Easier to Rebuild Than to Repair

"You name them and when they get sick, you nurse them back to health. [Now] servers are [treated] like cattle. You number them and when they get sick, you shoot them."

Bill Baker @ Microsoft



Almost all large-scale web operations do this



Easy horizontal scaling



Avoid disaster - restore service after a catastrophic failure
(years of undocumented and manual production changes)

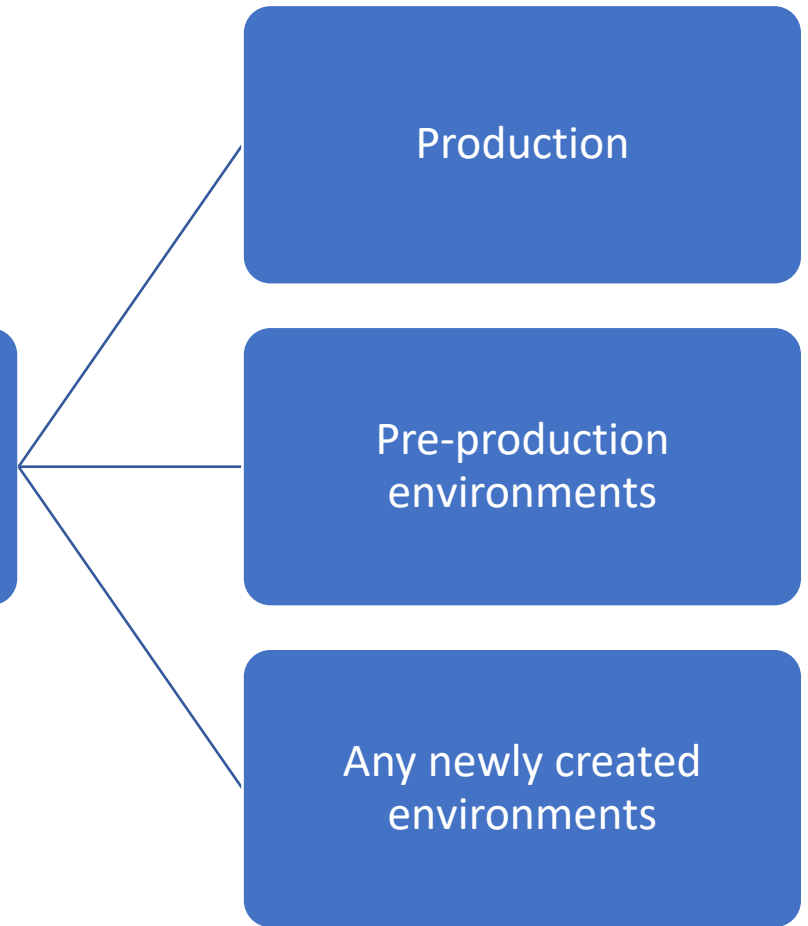
Ensure consistency

Changes (configuration changes, patching, upgrading, etc.) replicated automatically everywhere

Production

Pre-production environments

Any newly created environments



Ways to propagate configuration



Rely on automated configuration systems to ensure consistency (e.g., Puppet, Chef, Ansible, Salt, Bosh, etc.)



Use a service mesh or configuration management service to propagate runtime configuration (Istio, AWS Systems Manager Parameter Store etc.)



Create new virtual machines or containers from our automated build mechanism and deploy them into production, destroying the old ones or taking them out of rotation - **Immutable Infrastructure**

Prevent uncontrolled configuration

Disable	disable remote logins to production servers
Kill and replace	routinely kill and replace production instances => manually applied production changes are removed
Keep	Keep pre-production environments up to date (e.g., developers)

Modify Our Definition of Development “Done” to Include Running in Production-Like Environments



Ensure environments are being used in the daily work of Development



Development and QA are routinely integrating the code with production-like environments at increasingly frequent intervals throughout the project (mitigate the risks)



End of each development interval (or more frequently) we have integrated, tested, working, and potentially shippable code, demonstrated in a production-like environment



Successfully deployed and run code in production-like environments hundreds or even thousands of times



Eliminate an entire class of operational and security defects and architectural problems that are usually caught too late in the project to fix

Automated Tesing

A close-up photograph of a laboratory setting. A person wearing a white lab coat and a white nitrile glove is carefully placing a small, clear vial with a blue cap into a multi-well plate. The plate is part of a larger, light-colored laboratory instrument, possibly a microplate reader or a liquid handling robot. The background is blurred, showing other laboratory equipment and a clean, professional environment. The text "Menti: What are the benefits of automated tests?" is overlaid in white, sans-serif font across the center of the image.

Menti: What are the benefits
of automated tests?

Enable Fast and Reliable Automated Testing



developers learn about their mistakes fast



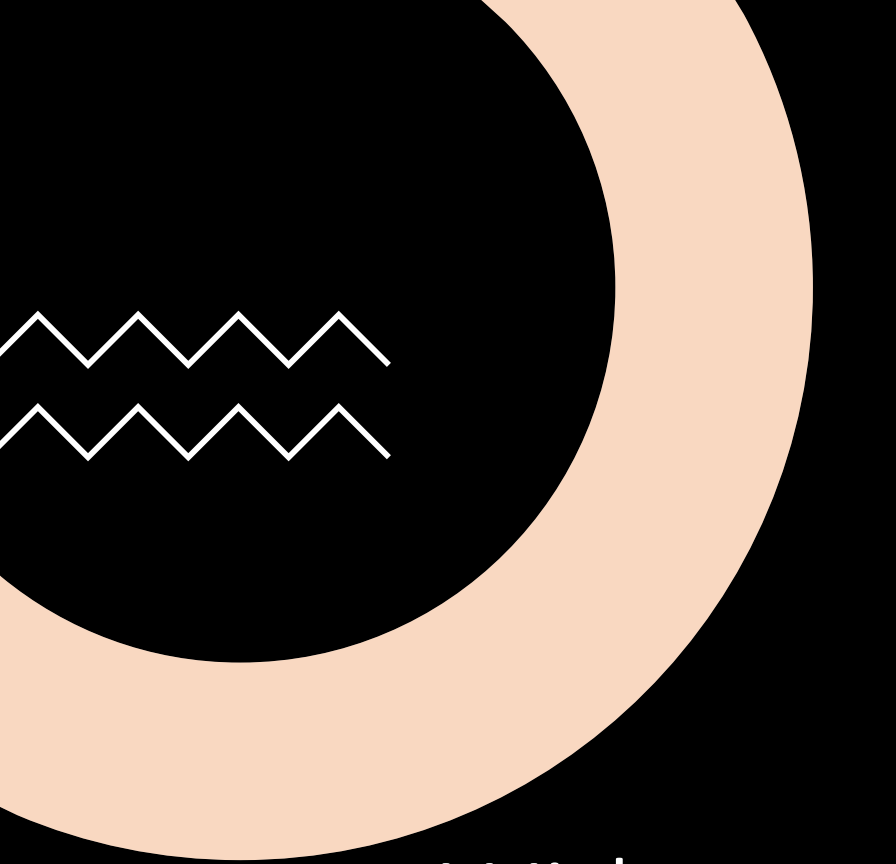
the link between cause and effect is still fresh



solving the problem is easier as opposed to requiring firefighting and archaeology



ability to learn from the mistake and integrate it into future work



“the more code we write, the more time and money is required to test our code—in most cases, this is a totally unscalable business model for any technology organization.”

Gary Gruver

Without
Automated
Testing



Continuously Build, Test, and Integrate Our Code and Environments



Have developers build automated tests as part of their daily work



Create a fast feedback loop that helps developers find problems early and fix them quickly when there are the fewest constraints



Ensures that all code checked into version control is automatically built and tested in a production-like environment



Assured that service is in a deployable and shippable state

Create automated build and test processes that run in dedicated environments



Build and test process can run all the time, independent of the work habits of individual engineers.



A segregated build and test process ensures that we understand all the dependencies required to build, package, run, and test our code



We can package our application to enable the repeatable installation of code and configurations into an environment

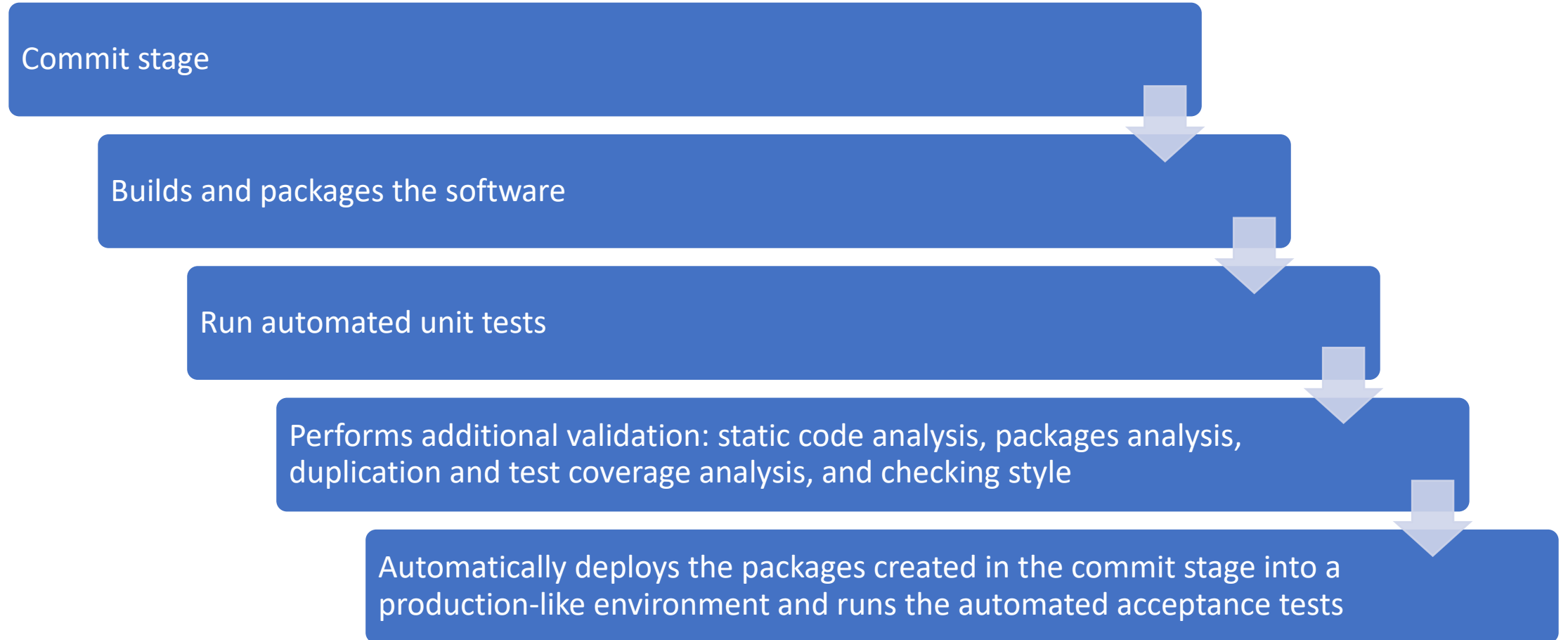


Instead of putting our code in packages, we may choose to package our applications into deployable containers

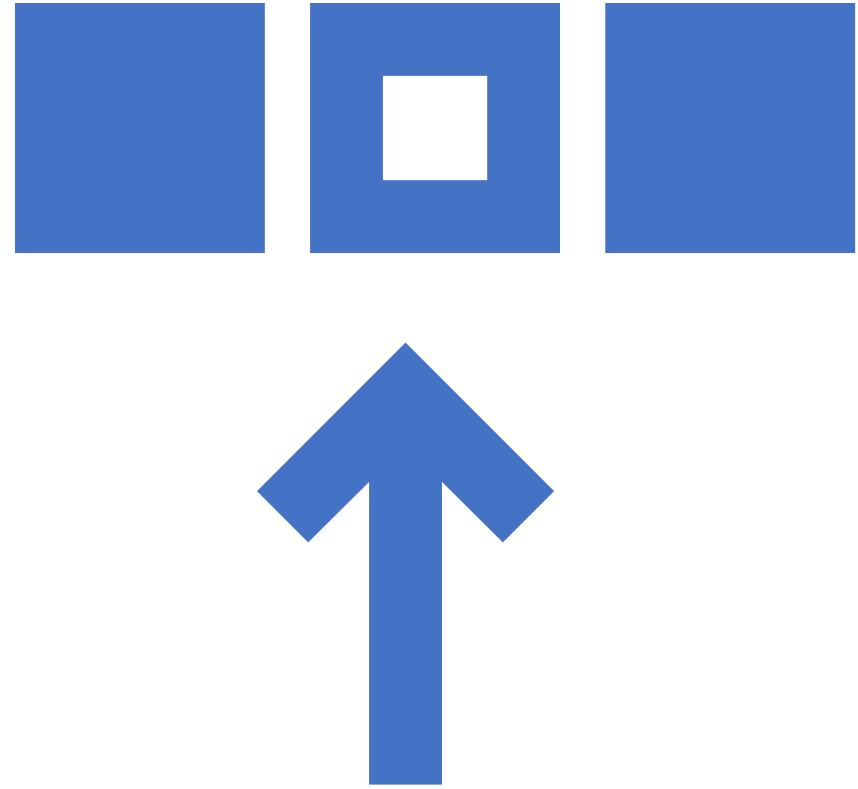


Environments can be made more production-like in a way that is consistent and repeatable

Deployment Pipeline



The goal of the deployment pipeline is to provide everyone in the value stream, especially developers, the fastest possible feedback that a change has taken us out of a deployable state.



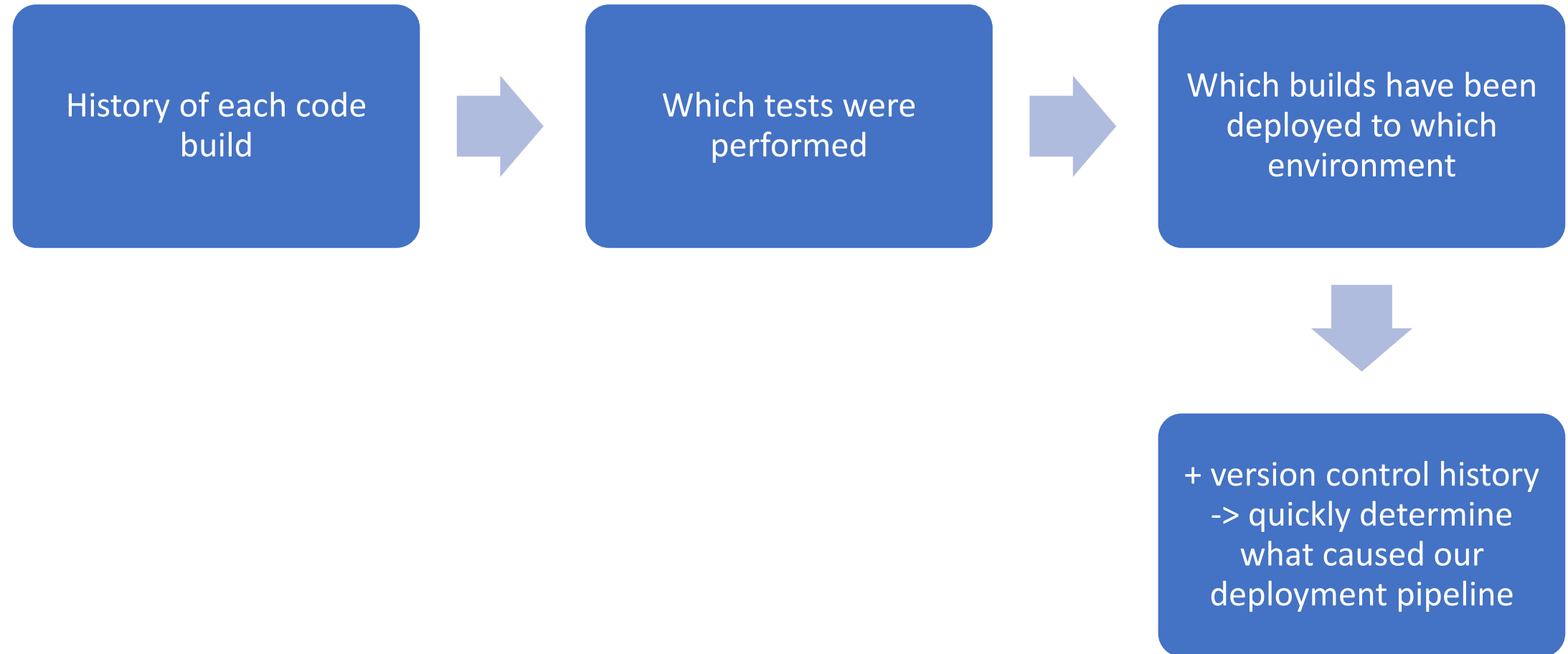
Package code only
once so that the
same packages are
used to deploy
code throughout
entire deployment
pipeline





Menti: What are the benefits
of deployment pipelines?

Deployment Pipeline Benefits



Tools

Jenkins

TeamCity

GitLab CI

Azure DevOps

GitHub Actions

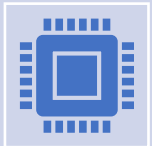
Continuous Integration



a comprehensive and reliable set of automated tests that validate we are in a deployable state



a culture that “stops the entire production line” when our validation tests fail



developers working in small batches on trunk rather than long-lived feature branches

Build a Fast and Reliable Automated Validation Test Suite

Slow and periodic feedback kills, especially for larger development teams



encourage developer to integrate the code at the end



big-bang integrations

Automated Tests Categories



Unit tests: test a **single** code module in **isolation**, providing assurance to the developer that their code operates as designed (**< 10 min**)



Acceptance tests: These typically test the application as a whole to provide assurance that a higher level of functionality operates as designed, and that regression errors have not been introduced (**hours**)



Integration tests: Integration tests are where we ensure that our application correctly interacts with other production applications and services, as opposed to calling stubbed out interfaces (**hours**)

Catch Errors as Early in Our Automated Testing as Possible



Tests run order

faster-running automated tests (e.g., unit tests)
slower-running automated tests (e.g., acceptance and integration tests),
manual testing



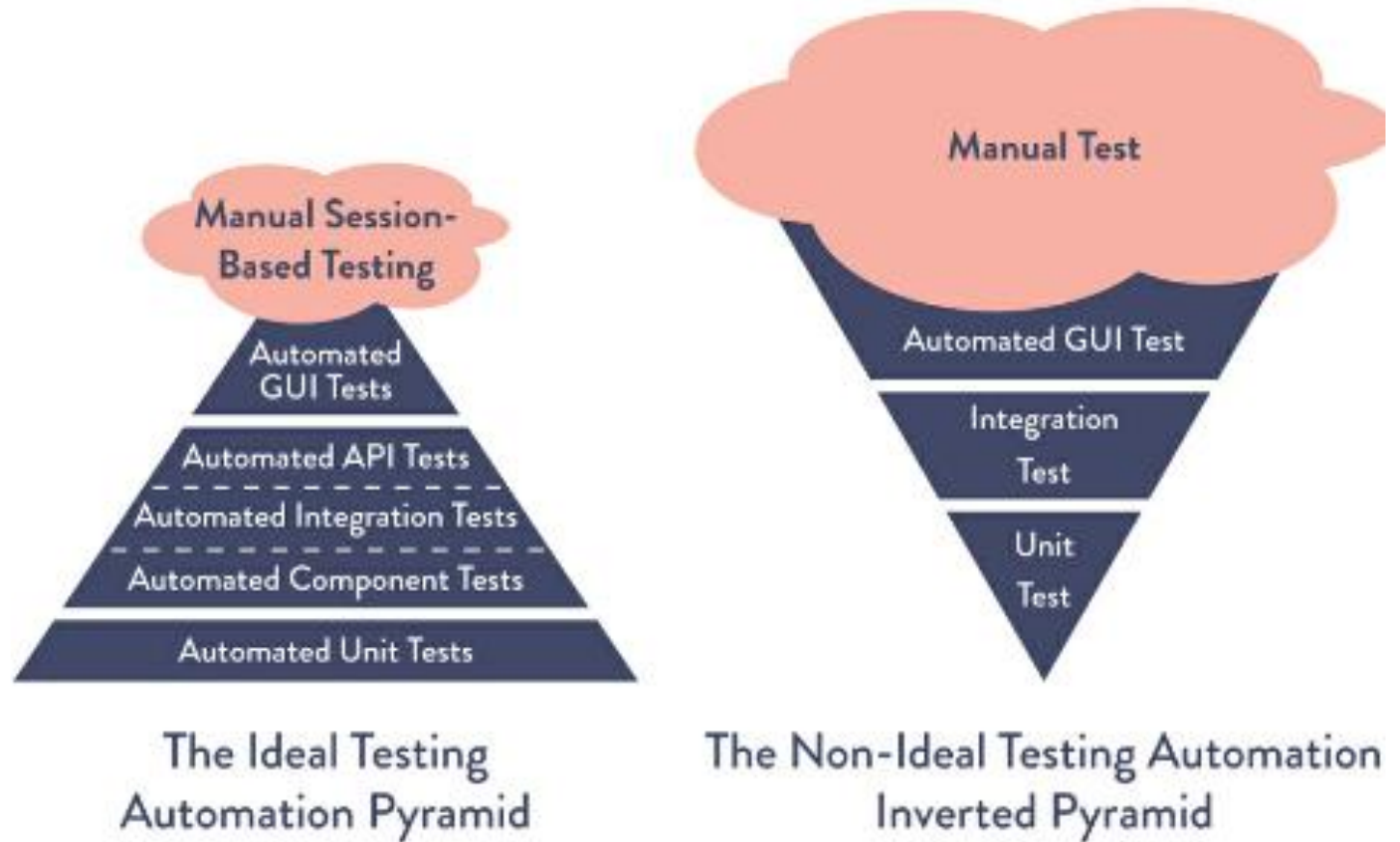
any errors should be found with the fastest category of testing possible



whenever an error is found with an acceptance or integration test, a unit test should be created that could find the error faster, earlier, and cheaper

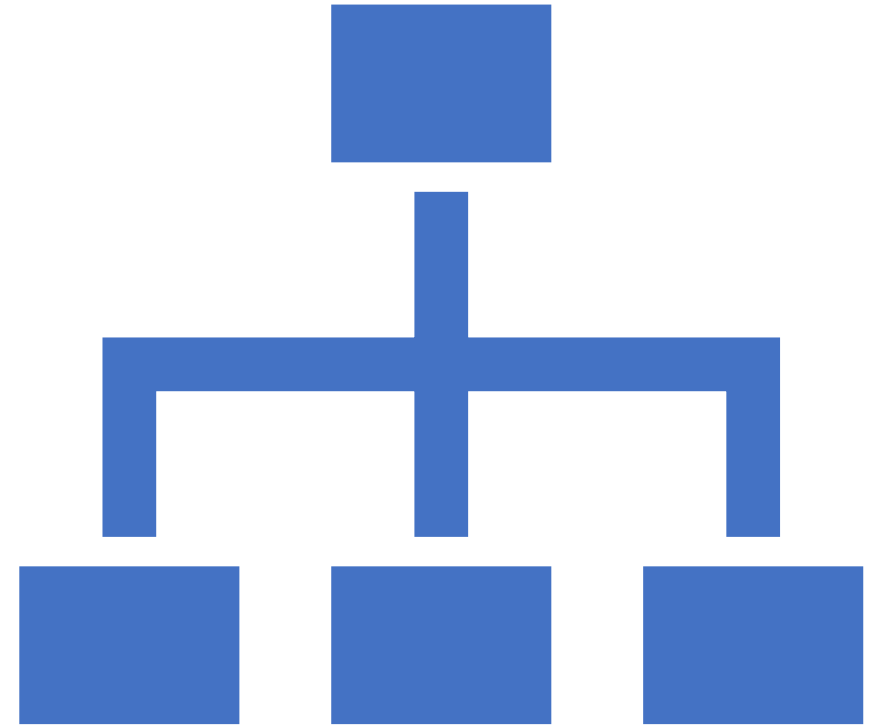
Ideal Testing Pyramid (Martin Fowler)

Ideal vs. Non-Ideal Testing Pyramids

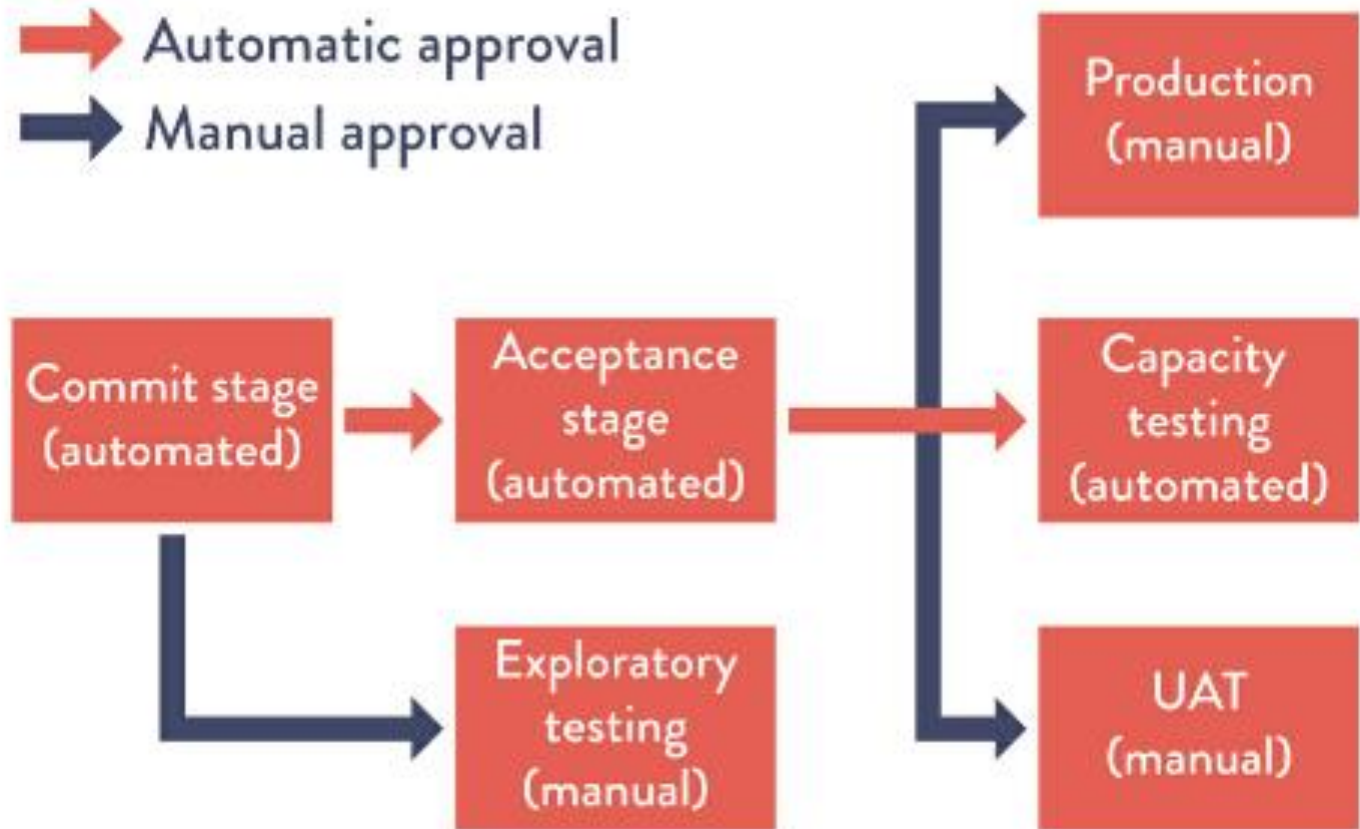


Source: Martin Fowler, "TestPyramid," MartinFowler.com.

If unit or acceptance tests are too difficult and expensive to write and maintain => architecture is too tightly coupled, where strong separation between our module boundaries no longer exists (or maybe never existed)

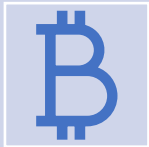


Ensure Tests
run Quickly
(In Parallel, If
Needed)



Source: Humble and Farley, Continuous Delivery, Kindle edition,
location 3868.

Write Our Automated Tests before We Write the Code (“Test-Driven Development”)



Ensure the tests fail. “Write a test for the next bit of functionality you want to add.” Check in.



Ensure the tests pass. “Write the functional code until the test passes.” Check in.



“Refactor both new and old code to make it well structured.” Ensure the tests pass. Check in again

Automate as Many of Our Manual Tests as Possible

To have humans executing tests that should be automated is a waste of human potential (Elisabeth Hendrickson)

Enable all testers (includes developers) to work on high-value activities that cannot be automated, such as exploratory testing or improving the test process itself

Unreliable tests that generate false positives create significant problems

Small number of reliable, automated tests are almost always preferable over a large number of manual or unreliable automated tests

Other Types of Tests should be Integrated



Performance Testing

upgrading a library caused 10x performance degradation

Database query-time grows non-linearly

Code change generates chattiness (10x)



Non-functional requirements

Environment built and configured properly

Pull Andon Cord When the Deployment Pipeline Breaks

build or automated tests fails => no new work is allowed to enter the system until the problem is fixed

if someone needs help to resolve the problem, they can bring in whatever help they need

team should be empowered to roll back the commit to get back into a green state

acceptance tests or performance tests failure, doesn't stop new work, they should be fixed immediately by developers and testers on call, and they will add new tests that run at an earlier stage in the deployment pipeline to catch any future regressions

The Essential Components of Automated Testing



Reliable: A failure signals a real defect, and when tests pass, developers are confident the code will run successfully in production.



Consistent: Each code commit should trigger a set of tests, providing feedback to developers.



Fast and reproducible: Tests should complete in ten minutes or less so developers can quickly reproduce and fix failures in personal environments.



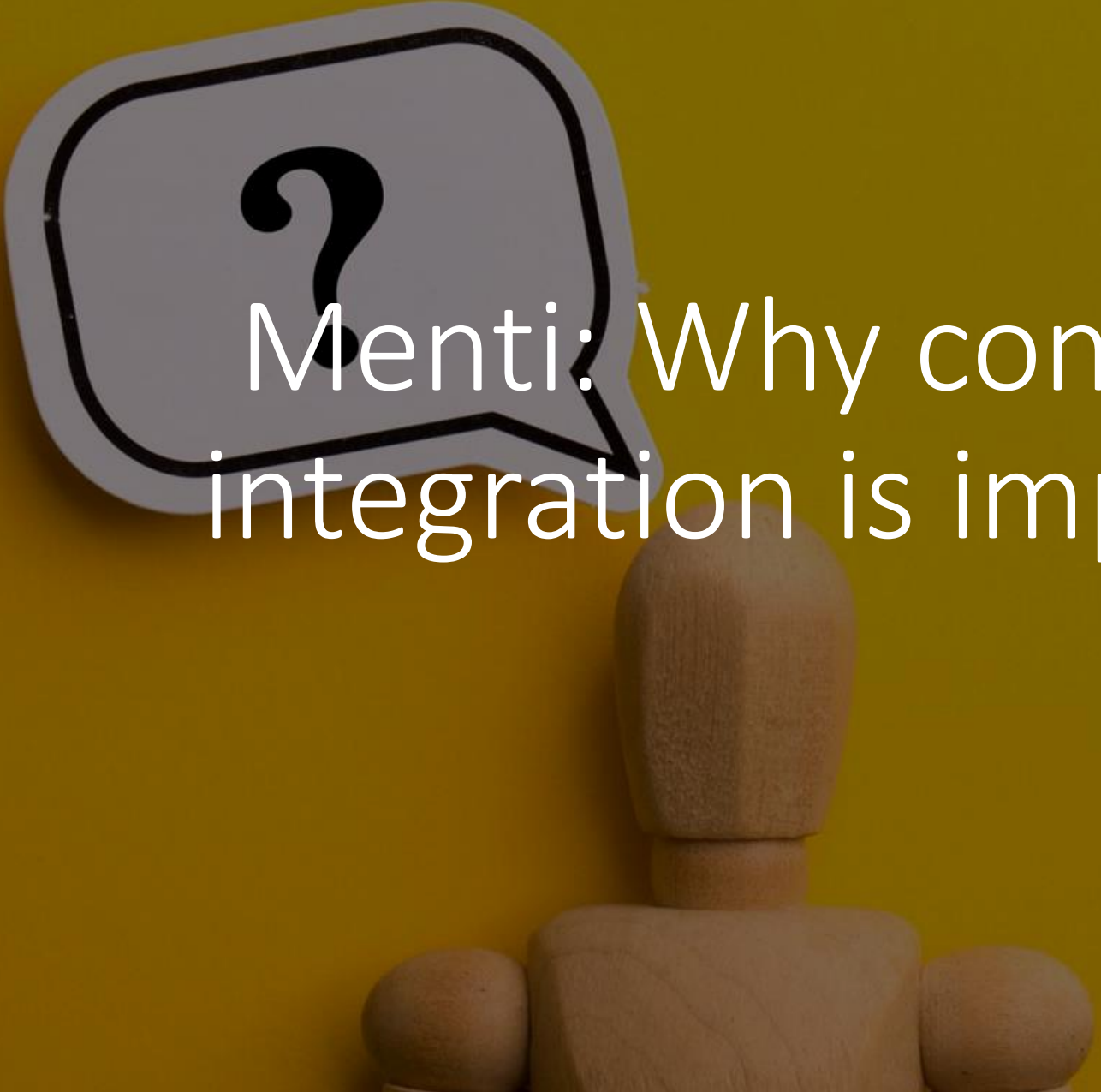
Inclusive: Testing shouldn't just be for testers, and the best outcomes are seen when developers practice test-driven development.

Curs 4



Continuous
Integration



A wooden figure is positioned at the bottom center of the frame. Above its head is a light gray speech bubble with a black outline and a black question mark inside. The background is a solid olive green color.

Menti: Why continuous
integration is important?

Enable and Practice Continuous Integration



The longer developers are allowed to work in their branches in isolation, the more difficult it becomes to integrate and merge everyone's changes back into trunk



Integrating those changes becomes exponentially more difficult as we increase the number of branches and the number of changes in each code branch



downward spiral: when merging code is painful, we tend to do it less often, making future merges even worse

Ways of working



Optimize for individual productivity: Every single person on the project works in their own private branch. Merging becomes a nightmare.



Optimize for team productivity: Everyone works in the same common area. There are no branches. Commits are simple, but each commit can break the entire project

Adopt Trunk-Based Development Practices

Developers check their code into trunk at least once per day



Detect merge problems when they are small easy and cheap to fix



Gated commits - before actually being merged into trunk deployment pipeline first confirms that:

submitted change will successfully
merge

build as expected

pass all the automated tests

Continuous Integration and Definition of Done

“At the end of each development interval, we must have integrated, tested, working, and potentially shippable code, demonstrated in a production-like environment, created from trunk using a one-click process, and validated with automated tests”



Integrate With Production
Environment

Automate and Enable Low-Risk Releases



By deferring production deployments, we accumulate ever-larger differences between the code to be deployed and what's running in production, increasing the deployment batch size



High risk of unexpected outcomes associated with the change, as well as the difficulty fixing them

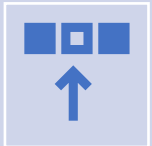
Automate Deployment Process

Packaging	code in ways suitable for deployment
Creating	preconfigured virtual machine images or containers
Automating	the deployment and configuration of middleware
Copying	packages or files onto production servers
Restarting	servers, applications, or services
Generating	configuration files from templates
Running	automated smoke tests to make sure the system is working and correctly configured
Running	testing procedures
Scripting and automating	database migrations

Requirements for Deployment Pipeline



Deploying the same way to every environment

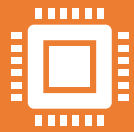


Smoke testing our deployments



Ensure we maintain consistent environments

Enable Automated Self-Service Deployments



Developers' ability to self-deploy code into production, to quickly see happy customers when their feature works, and to quickly fix any issues without having to open up a ticket with Operations has diminished over the last decade (**control and oversight**)



Alternative

Separation of duties through code reviews

Automated test suites

Self-serviced deployments through an automated system

Phases of the deployment automation

1

Build: create packages from version control that can be deployed to any environment

2

Test: run any or all of the automated test suite on workstation or on test systems.

3

Deploy: Anybody should be able to deploy these packages to any environment where they have access, executed by running scripts that are also checked into version control.

What Should Provide Deployment Automation

Ensure	Show	Push-Button	Record Automatically	Smoke Test	Feedback
that packages are suitable for deployment into production	the readiness of production environments at a glance	provide a push-button, self-service method for packaged code to be deployed	for auditing and compliance purposes, which commands, who authorized, and what the output was; the hash of all binaries deployed and commit	to ensure the system is operating correctly and the configuration settings are correct	so that developers can quickly determine whether their deployment was successful



Decouple Deployment
from Release

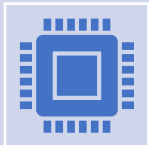
Decouple Deployments from Releases



Deployment is the installation of a specified version of software to a given environment



Release is when we make a feature (or set of features) available to all our customers or a segment of customers

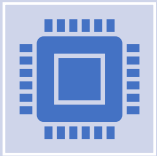


Code and environments should be architected in such a way that the release of functionality **does not require changing** the application code.

Menti: How can we
decouple deployment from
release?



Release patterns

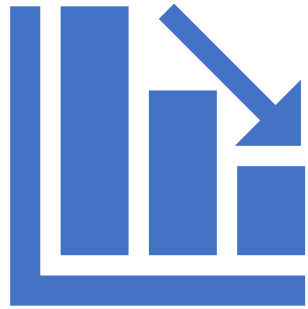


Environment-based release patterns: This is where we have two or more environments that we deploy into, but only one environment is receiving live customer traffic (e.g., by configuring our load balancers). Examples: blue-green deployments, canary releases, and cluster immune systems

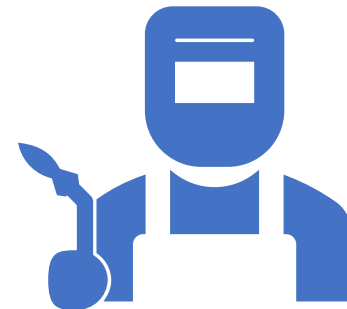


Application-based release patterns: This is where we modify our application so that we can selectively release and expose specific application functionality by small configuration changes. Examples: dark launching using feature flags

Environment-base Release Patterns



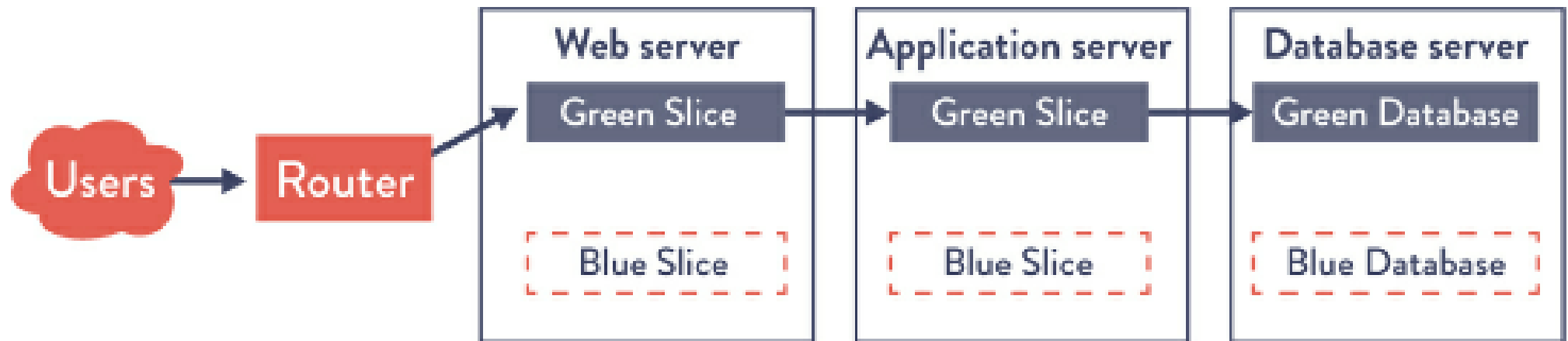
No longer have to perform deployments in the middle of the night or on weekends to lower the risk of negatively impacting customers



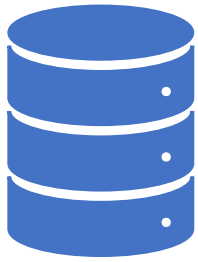
Deployments during typical business hours, enabling Ops to finally have normal working hours, just like everyone else

The Blue-Green Deployment Pattern

- Two production environments
- One is serving customer traffic
- Deployment happens to inactive environment
- Release and rollback happens through changing how traffic is routed



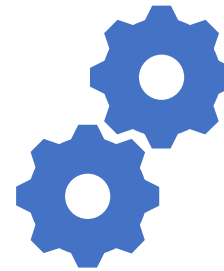
Dealing with Database Changes



Create two databases: Each version of the application has its own database.

Release => blue database into read-only mode, perform a backup of it, restore onto the green database, and finally switch traffic to the green environment

The problem – if we need to roll back to the blue version, we can potentially lose transactions if we don't manually migrate them from the green version first

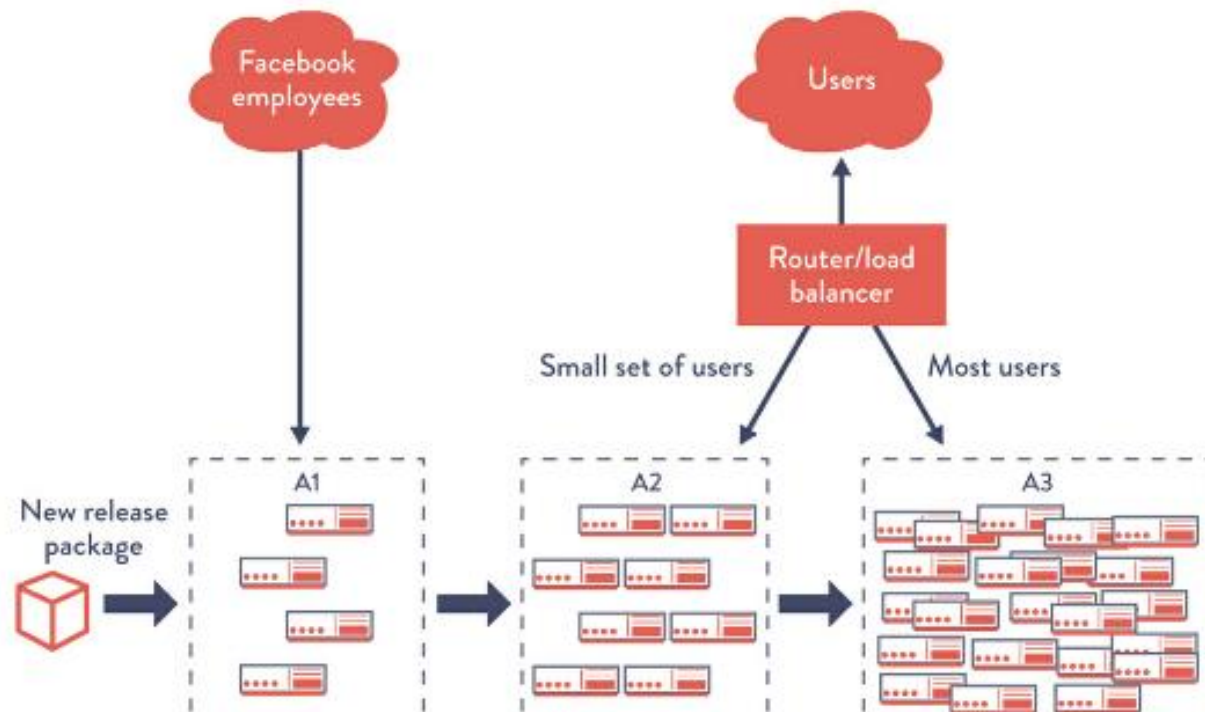


Decouple database changes from application changes:

make only additive changes to database; never mutate existing database objects

make no assumptions in the application about which database version will be in production

Canary Release



- Perform release
- Monitor
- Rollback if errors
- Deploy to next environment if OK

Cluster immune system



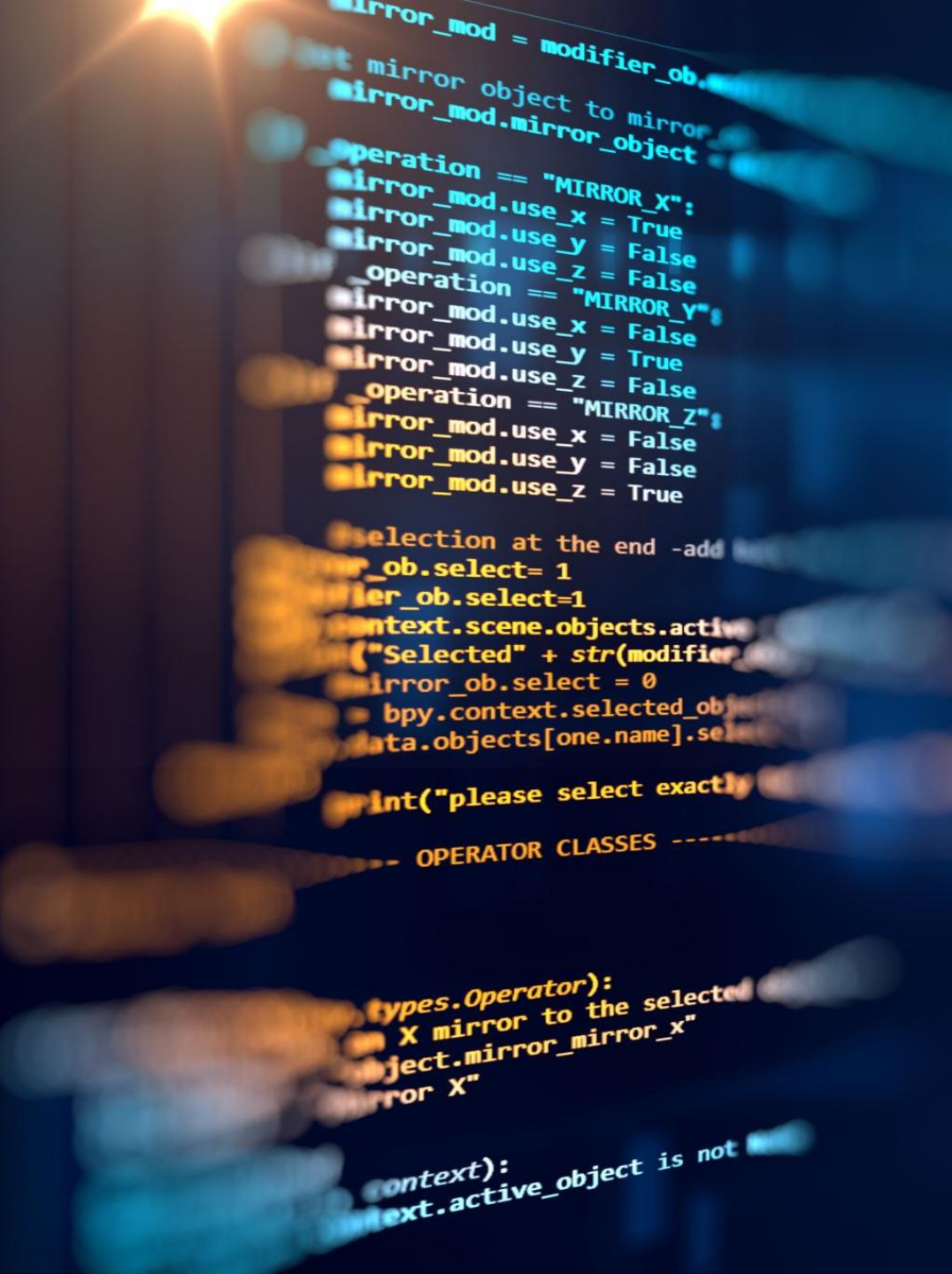
Links production monitoring system with release process, by automating the rollback of code when the user-facing performance of the production system deviates outside of a predefined expected range



Benefits

protect against defects that are hard to find through automated tests

reduce the time required to detect and respond to the degraded performance



```
mirror_mod = modifier_ob.  
# Set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
# selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly  
-- OPERATOR CLASSES ----  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
context):  
context.active_object is not
```

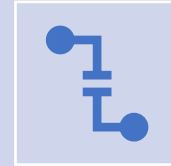
Application-Based Patterns to Enable Safer Releases - Feature Toggles

- A mechanism to selectively enable and disable features without requiring a production code deployment
- Feature toggles can also control which features are visible and available to specific user segments
- Usually implemented by wrapping application logic or UI elements with a conditional statement
- Can be stored in application configuration file, a directory service or a web service specifically designed to manage feature toggling

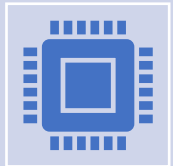
Benefits of feature toggles



Roll back easily – switching off one particular stakeholder's features is usually much easier than rolling back an entire release



Gracefully degrade performance - increase the number of users we serve by reducing the level of functionality delivered



Increase our resilience through a service-oriented architecture - If we have a feature that relies on another service that isn't complete yet/ or fails, we can still deploy our feature into production but hide it behind a feature toggle



Perform A/B testing: also enable product teams to run experiments to test new features and see their impact on business metrics

Perform Dark Launches

Deploy

- features into production without making them accessible to users

Perform

- testing of new functionality while it is still invisible to customers

Have X%

- of our online users make invisible calls to a new feature scheduled to be launched to see how our new feature behaves under load

Increase

- After problems are found and fixed, simulated load can be progressively increased by increasing the frequency and number of users exercising the new functionality

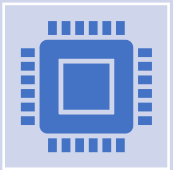
Roll out

- progressively roll out the feature to small segments of customers, halting the release if any problems

Continuous Delivery vs Continuous Deployment



Continuous Delivery - when all developers are working in small batches on trunk, or everyone is working off trunk in short-lived feature branches that get merged to trunk regularly, and when trunk is always kept in a releasable state, and when we can release on demand at the push of a button during normal business hours



Continuous Deployment - When we are deploying good builds into production on a regular basis through self-service—which typically means that we are deploying to production at least once per day per developer, or perhaps even automatically deploying every change a developer commits



Architect for Low-Risk
Releases

Architect for Low-Risk Releases



The principle of evolutionary architecture—architecture of “any successful product or organization will necessarily evolve over its life cycle” - Jez Humble



“Both eBay and Google are each on their fifth entire rewrite of their architecture from top to bottom” – Randy Shoup



Challenge is how to keep migrating from the architecture we have to the architecture we need

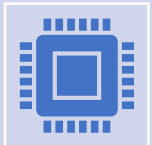
An Architecture that Enables Productivity, Testability, and Safety



Loosely coupled architecture with well-defined interfaces that enforce how modules connect with each other promotes productivity and safety



Enables small, productive teams that are able to make small changes that can be safely and independently deployed

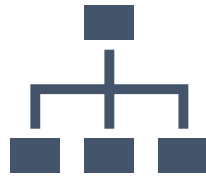


Enables easier testing of services and the creation of contracts and SLAs between teams.

Architectural Archetypes: Monoliths vs. Microservices



Most successful business started as monoliths



Monoliths

extremely successful at helping them achieve product/market fit

Put businesses at risk of organizational failure once they have to operate at scale



Evolved to Service Oriented / Microservices architecture

Monolithic v1 (all functionality in one application)

- Simple at first
- Low inter-process latencies
- Single code base; one deployment unit
- Resource efficient at small scales

- Coordination overhead increases as team grows
- Poor enforcement of modularity
- Poor scaling
- All-or-nothing deploy (downtime, failures)
- Long build times

- Simple at first
- Join queries are easy
- Single schema, deployment
- Resource efficient at small scales

Monolithic v2 (sets of monolithic tiers: “front end presentation,” “application server,” “database layer”)

- Tendency for increased coupling over time
- Poor scaling and redundancy (all-or-nothing, vertical only)
- Difficult to tune properly
- All-or-nothing schema management

- Each unit is simple
- Independent scaling and performance
- Independent testing and deployment
- Can optimally tune performance (caching, replication, etc.)

Microservice (modular, independent, graph relationship vs. tiers, isolated persistence)

- Many cooperating units
- Many small repos
- Requires more sophisticated tooling and dependency management
- Network

Lesson Learned from Amazon



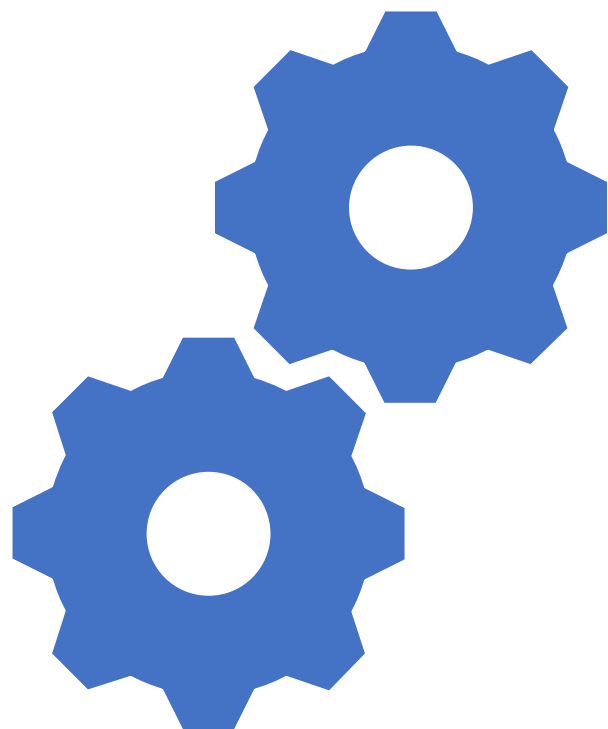
Lesson 1: When applied rigorously, strict service orientation is an excellent technique to achieve isolation; you achieve a level of ownership and control that was not seen before.



Lesson 2: Prohibiting direct database access by clients makes performing scaling and reliability improvements to your service state possible without involving your clients.



Lesson 3: Development and operational process greatly benefits from switching to service orientation. The services model has been a key enabler in creating teams that can innovate quickly with a strong customer focus. Each service has a team associated with it, and that team is completely responsible for the service—from scoping out the functionality to architecting, building, and operating it.



How to evolve an
architecture

Big-Bang approach

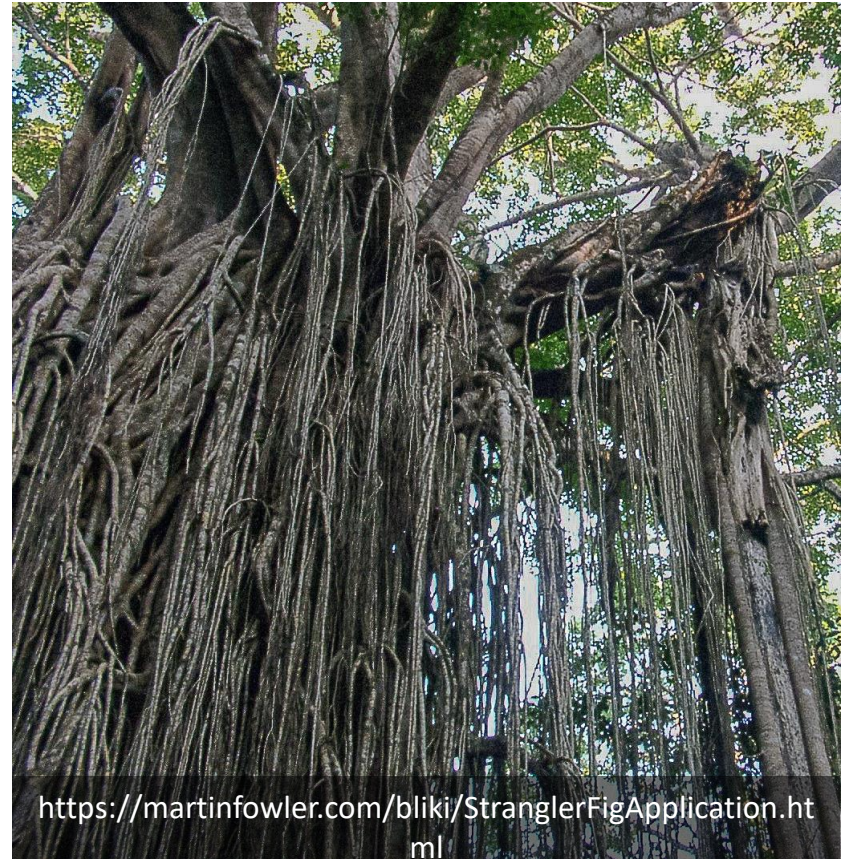
Missing important
functionality

Bugs

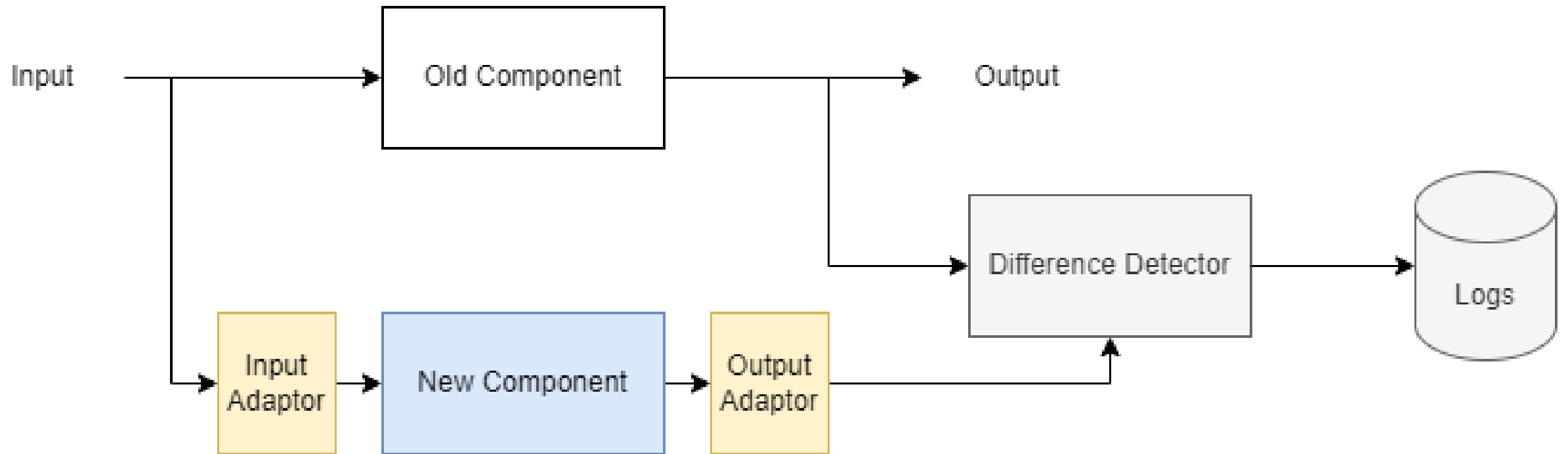
Performance problems

Strangler Fig Application Pattern

- instead of “ripping out and replacing” old services with architectures that no longer support our organizational goals, we put the existing functionality behind an API and avoid making further changes to it. All new functionality is then implemented in the new services that use the new desired architecture, making calls to the old system when necessary
- useful for helping migrate portions of a monolithic application or tightly coupled services to one that is more loosely coupled
- create quick wins and deliver early incremental value before continuing to iterate



Build a new component



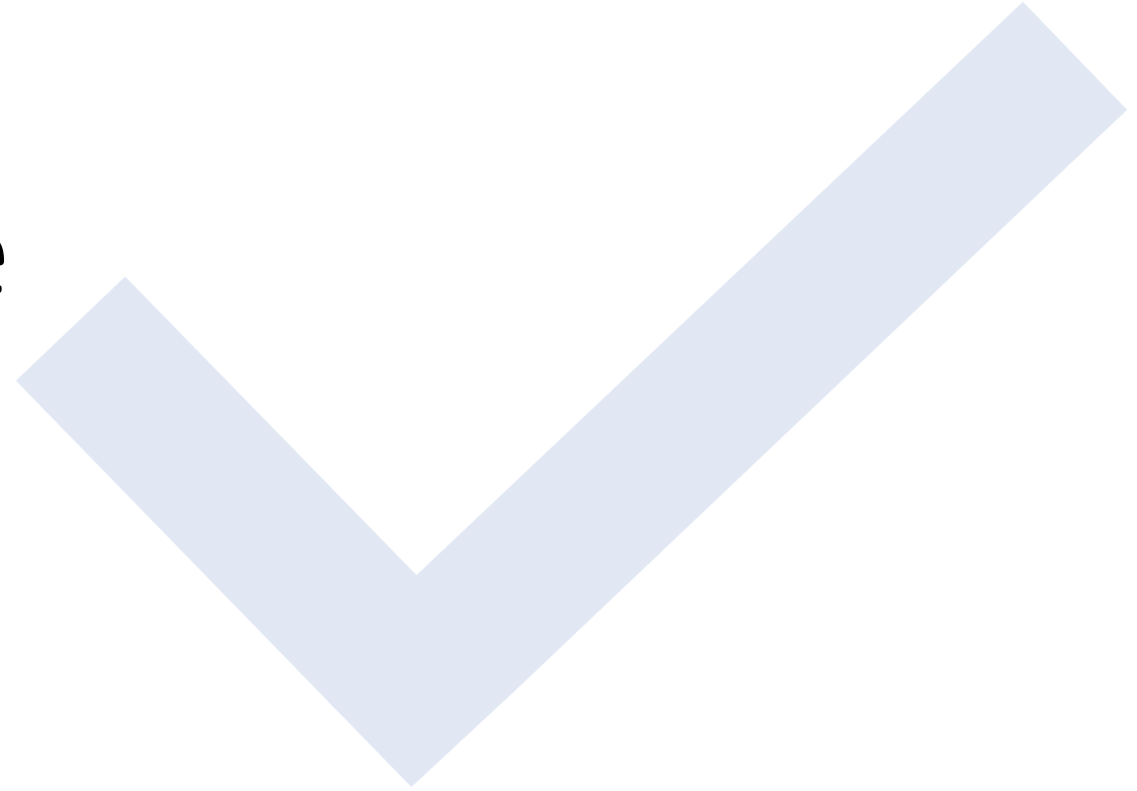
What to check

Functional and Non-Functional Testing

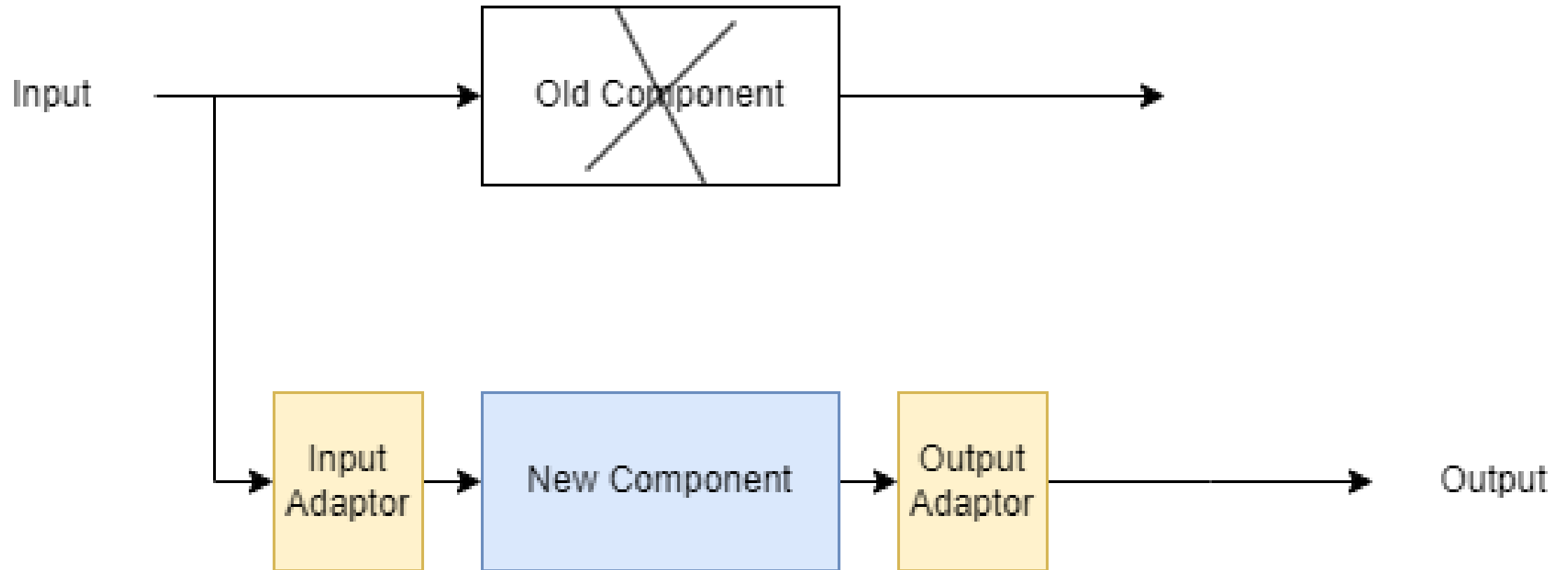




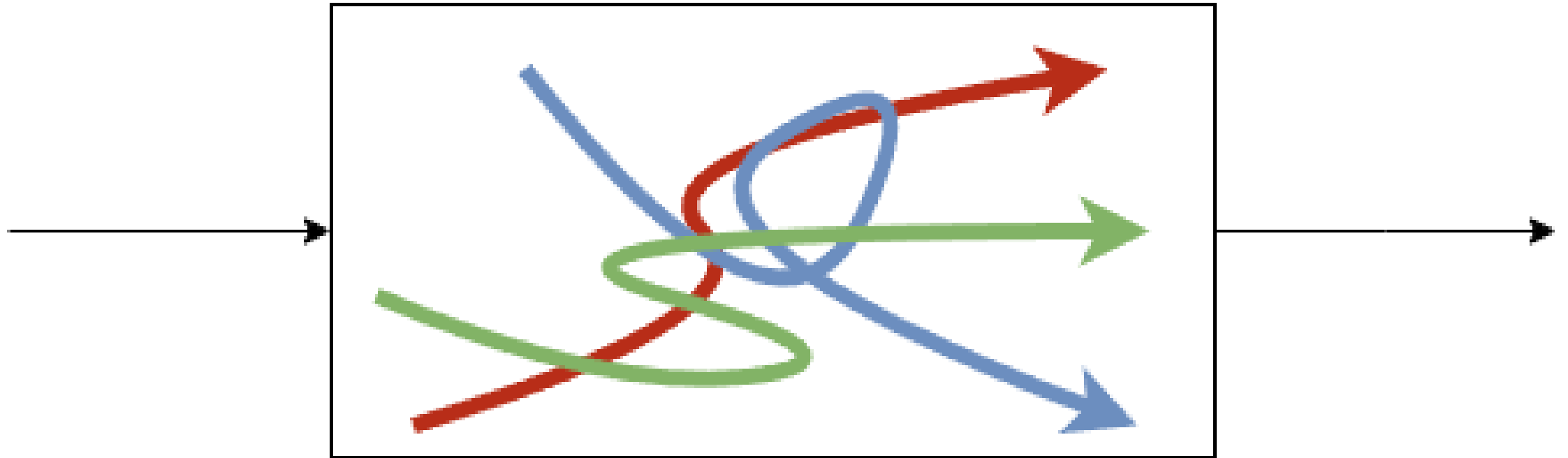
Online vs Offline Checks



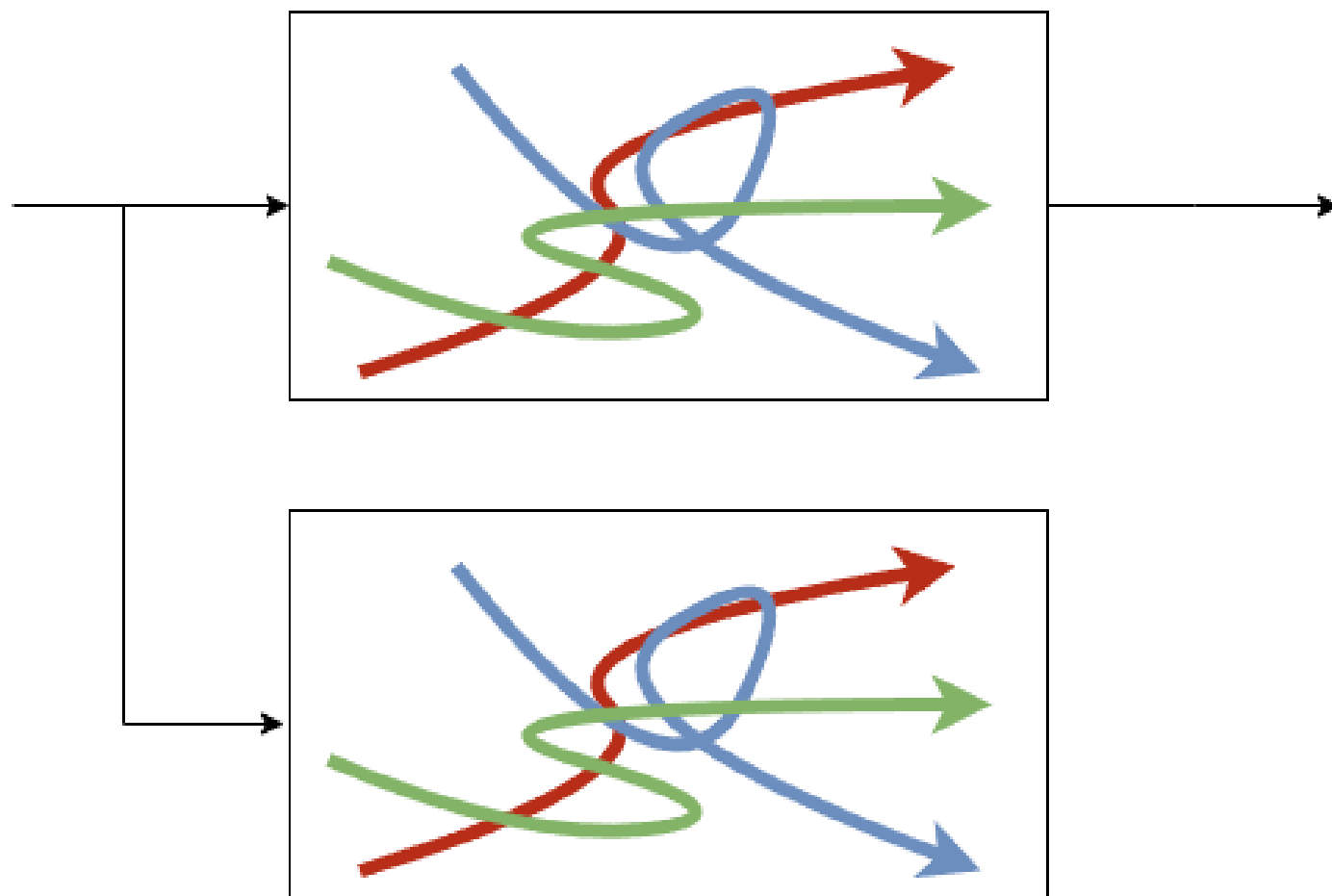
Enable the new component



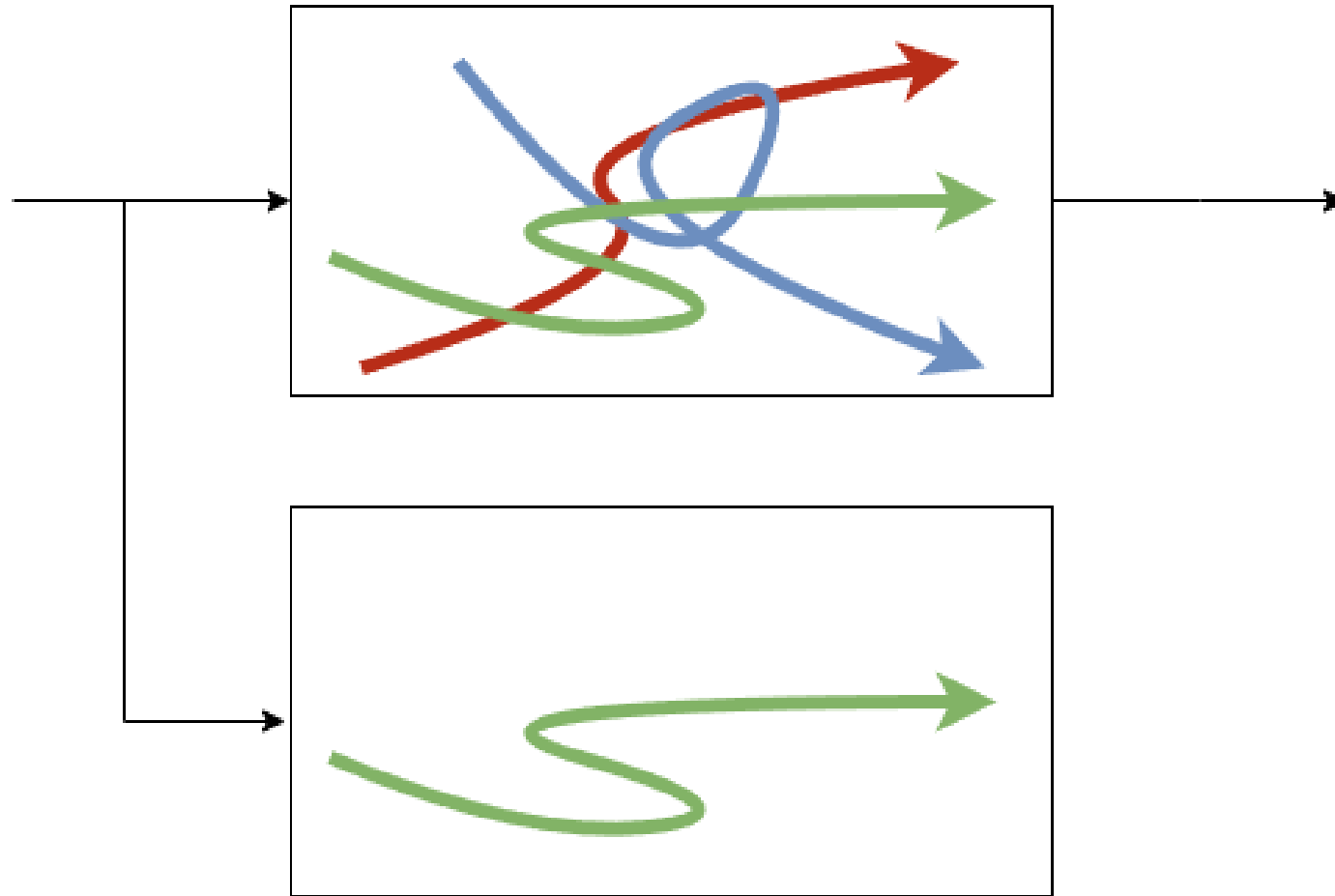
Decouple a component



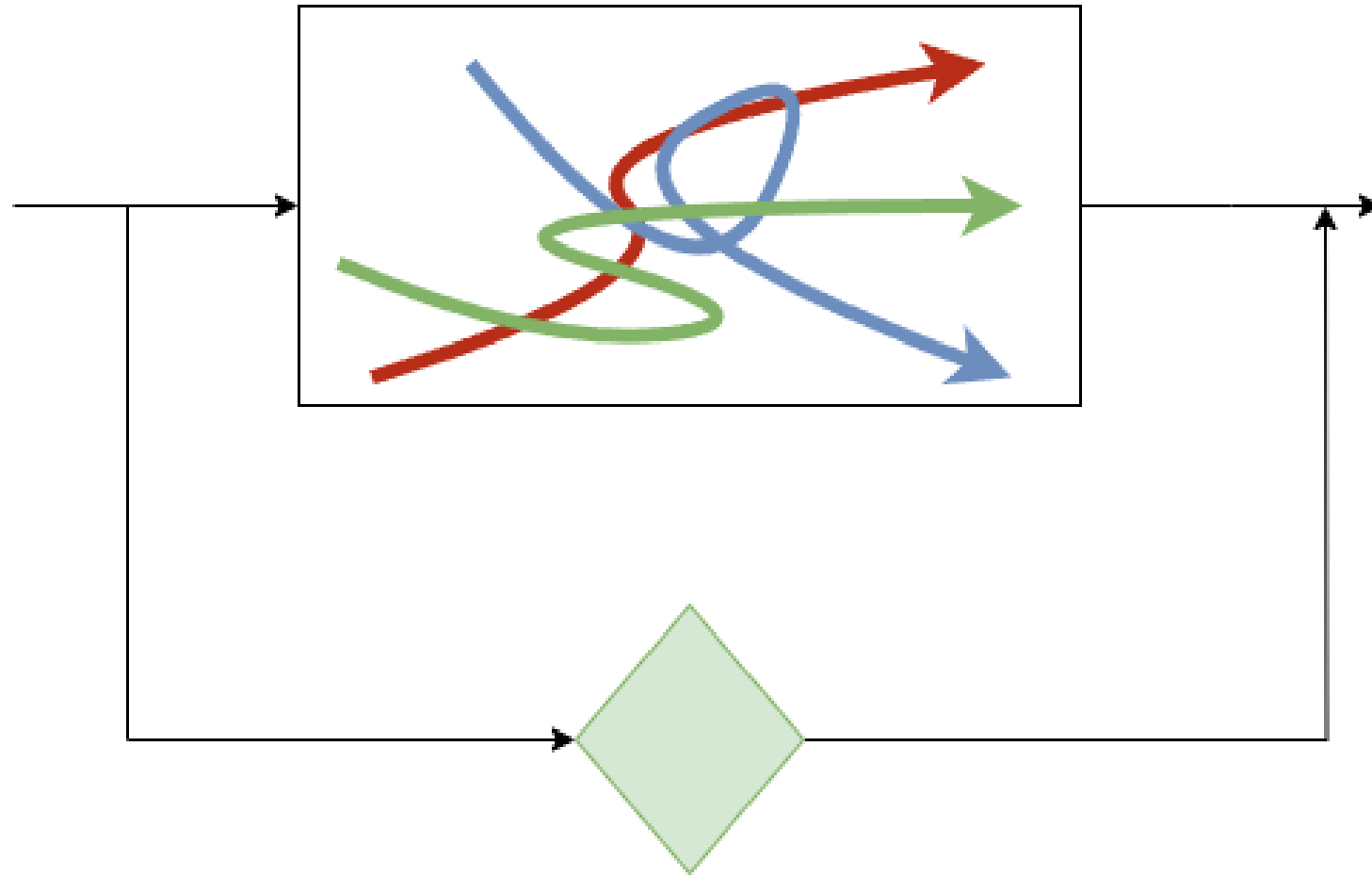
Replicate component



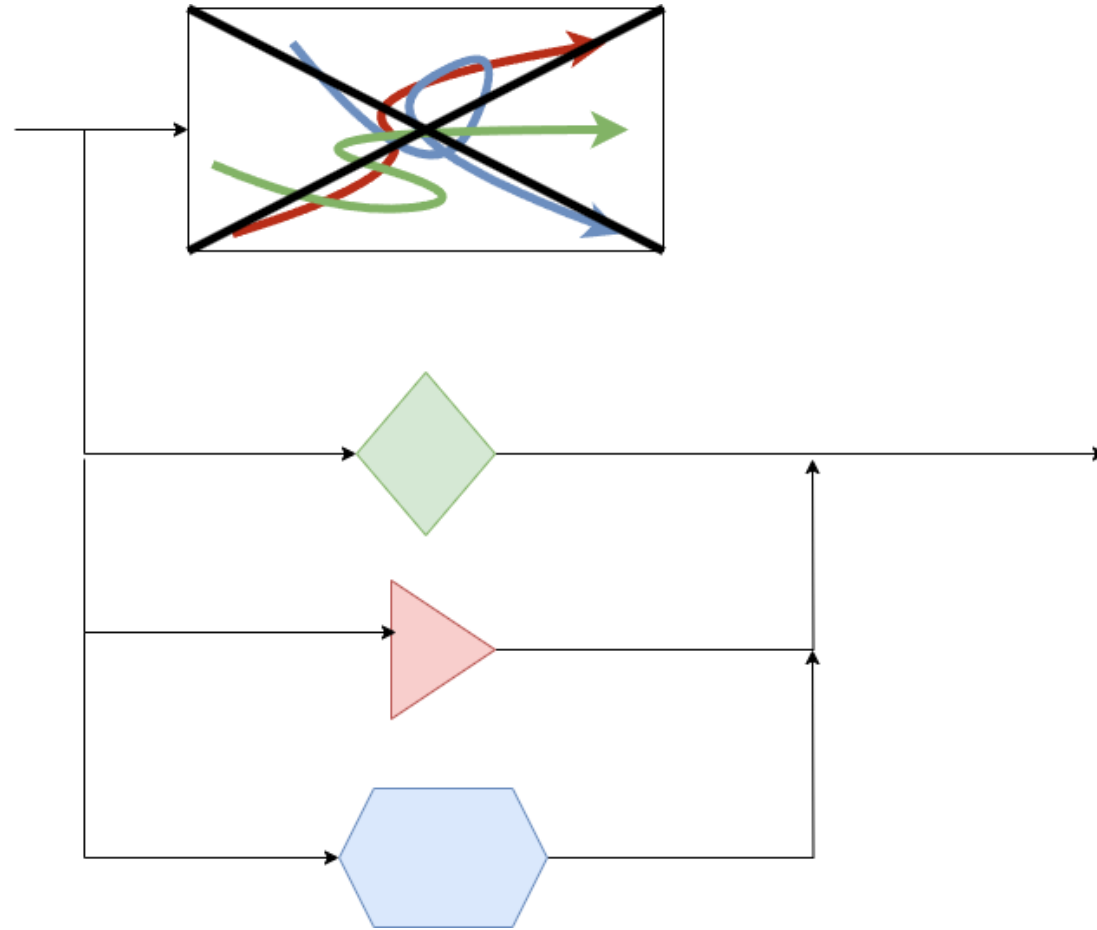
Remove other functionalities



Give the component the structure you want



Continue with the rest of the functionalities





Thank you

